

NBA 5th Player Prediction

Emily Lai - 100825007
Jeremy Mark Tubongbanua - 100849092
Natasha Naorem - 100845321

Group 6

The Problem

Input features:

- season (e.g. 2007)
- home_team (e.g. "IND")
- away_team (e.g. "BOS")
- List of 4 home players
- List of 5 away players

Output feature:

- 5th player

Our goal is to develop a model that takes these input features, and makes a prediction on who the 5th player that will optimize the outcome.

Pre-Processing the Data

In order to process the data, we need to “**index**” the data, which means taking the values from our data and transform them into indices.

Our goal was to create

- **games.txt** - list of all unique games
- **names.txt** - list of all unique names
- **seasons.txt** - list of all unique seasons
- **teams.txt** - list of all unique teams

Once everything was indexed..

And convert the existing columns from `home\_0, ... home\_4` to a single `home\_players` list of the 5 players. We named this dataframe as **mapped_data.csv**

Then remove a player for each row and create a **player_removed** column, ultimately multiplying the number of rows by 5. We named this dataframe as **mapped_expanded.csv**

Input features:

- season (e.g. 2007)
- home_team (e.g. “IND”)
- away_team (e.g. “BOS”)
- List of 4 home players
- List of 5 away players

Output feature:

- 5th player

Pre-Processing the Data

create_data.py (208 lines)

Goal: Encode the Strings into Dictionaries..

“Dictionary Encoding”

Running **create_data.py** (our data preprocessor)

Input: aggregated *matchups-20*.csv*

Output: games.txt, names.txt, seasons.txt, teams.txt

Additional Outputs: mapped_data.csv and
mapped_expanded.csv

```
create_data.py > ...
1 # --- Imports ---
2 import os
3 import glob
4 import pandas as pd
5 import requests
6 import zipfile
7 import random
8
9 # --- Download and extract data ---
10 if os.path.exists("data"):
11     print("Data directory already exists, skipping download and extraction.")
12 else:
13     url = "https://github.com/JeremyTubongbanua/sofe4620u-nba/raw/trunk/data.zip"
14
15     response = requests.get(url)
16     if response.status_code == 200:
17         with open("data.zip", "wb") as f:
18             f.write(response.content)
19
20         with zipfile.ZipFile("data.zip", 'r') as zip_ref:
21             zip_ref.extractall()
22
23     extracted_files = []
```

Output of create_data.py

matchups-20*.csv

- Total # Matches = 236 912
- Total Unique Names = 1044
- Total Unique Games = 10,828
- Total Unique Teams = 35
- Total Unique Seasons = 9
- Average # Matches / 9 Seasons = 26, 323.56

```
Total unique names in matchups-20*.csv: 1044
Total unique names now with NBA_test.csv: 1084
Total unique names: 1084 | First 5 names: ['A.J. Price', 'Aaron Brooks', 'Aaron Gordon', 'Aaron Gray', 'Aaron McKie']
Total unique games: 10828 | First 5 games: ['200610310LAL', '200610310MIA', '200611010BOS', '200611010CHA', '200611010CLE']
Total unique teams in matchups-20*.csv: 35
Total unique teams now with NBA_test.csv: 35
Total unique teams: 35 | First 5 teams: ['ATL', 'BOS', 'BRK', 'CHA', 'CHI']
Total unique seasons in matchups-20*.csv: 9
Total unique seasons now with NBA_test.csv: 10
Total unique seasons: 10 | First 5 seasons: ['2007', '2008', '2009', '2010', '2011']
```

Pre-Processing the Data

games.txt

```
data > data.table::fread("games.txt")
# A tibble: 10,828 x 2
#   game_id team
#   <int> <chr>
#1  200610310 LAL
#2  200610310 MIA
#3  200611010 BOS
#4  200611010 CHA
#5  200611010 CLE
#6  200611010 DET
#7  200611010 GSW
#8  200611010 MEM
```

10,828 unique games

names.txt

```
data > data.table::fread("names.txt")
# A tibble: 1,044 x 2
#   name_id name
#   <int> <chr>
#1    1 A.J. Price
#2    2 Aaron Brooks
#3    3 Aaron Gordon
#4    4 Aaron Gray
#5    5 Aaron McKie
#6    6 Aaron Williams
#7    7 Acie Law
#8    8 Adam Morrison
#9    9 Adonal Foyle
```

1,044 unique names

seasons.txt

```
data > data.table::fread("seasons.txt")
# A tibble: 9 x 2
#   season_id season
#   <int> <chr>
#1     1 2007
#2     2 2008
#3     3 2009
#4     4 2010
#5     5 2011
#6     6 2012
#7     7 2013
#8     8 2014
#9     9 2015
```

9 unique seasons

teams.txt

```
data > data.table::fread("teams.txt")
# A tibble: 35 x 2
#   team_id team
#   <int> <chr>
#1    12 HOU
#2    13 IND
#3    14 LAC
#4    15 LAL
#5    16 MEM
#6    17 MIA
#7    18 MIL
#8    19 MIN
```

35 unique teams

Pre-Processing the Data

Assumption: Order of players in a team do not matter. (lebron, luka, jeremy) == (luka, jeremy, lebron).
We weren't given any information on if the column position was a particular position in basketball (e.g. middle)

(1): Read the home teams and away teams

home_0	home_1	home_2	home_3	home_4
Ben McLemore	Darren Collison	DeMarcus Cousins	Jason Thompson	Rudy Gay
Ben McLemore	Darren Collison	DeMarcus Cousins	Jason Thompson	Rudy Gay
Carl Landry	Darren Collison	DeMarcus Cousins	Nik Stauskas	Rudy Gay
Carl Landry	Darren Collison	DeMarcus Cousins	Derrick Williams	Nik Stauskas
Carl Landry	Derrick Williams	Nik Stauskas	Ramon Sessions	Reggie Evans

away_0	away_1	away_2	away_3	away_4
Andrew Bogut	Draymond Green	Harrison Barnes	Klay Thompson	Stephen Curry
Draymond Green	Festus Ezeli	Harrison Barnes	Klay Thompson	Stephen Curry
Andre Iguodala	Festus Ezeli	Klay Thompson	Marreese Speights	Stephen Curry
Andre Iguodala	Festus Ezeli	Klay Thompson	Marreese Speights	Stephen Curry
Andre Iguodala	Festus Ezeli	Klay Thompson	Marreese Speights	Stephen Curry

(3): Create a column that has a sorted list of indices. Sorted lists because this ensures that the model doesn't see the same combination of players differently.

home_players	away_players
[88, 241, 256, 476, 897]	[51, 307, 396, 617, 958]
[88, 241, 256, 476, 897]	[307, 355, 396, 617, 958]
[138, 241, 256, 785, 897]	[43, 355, 617, 713, 958]
[138, 241, 256, 277, 785]	[43, 355, 617, 713, 958]
[138, 277, 785, 843, 859]	[43, 355, 617, 713, 958]

(2): Create a list of unique names

Total unique names: 1084

```
data > names.txt
1 A.J. Price
2 Aaron Brooks
3 Aaron Gordon
4 Aaron Gray
5 Aaron McKie
6 Aaron Williams
7 Acie Law
8 Adam Morrison
9 Adonal Foyle
```

Pre-Processing the Data

After preprocessing the data, this is what it looks like. Strings are converted to indices. This is **mapped_data.csv**

game	season	home_team	away_team	starting_min	outcome	home_players	away_players
9610	8	29	10	0	-1	[88, 241, 256, 476, 897]	[51, 307, 396, 617, 958]
9610	8	29	10	7	-1	[88, 241, 256, 476, 897]	[307, 355, 396, 617, 958]
9610	8	29	10	8	-1	[138, 241, 256, 785, 897]	[43, 355, 617, 713, 958]
9610	8	29	10	9	-1	[138, 241, 256, 277, 785]	[43, 355, 617, 713, 958]
9610	8	29	10	10	-1	[138, 277, 785, 843, 859]	[43, 355, 617, 713, 958]

Next, we remove one player at a time from home_players, and add it to `player\_removed` column.

home_team = **[88, 241, 256, 476, 897]** becomes

- (home_players, player_removed)
- [88,241,256,476], 897
- [88, 241, 256, 897], 476
- [88, 241, 476, 897], 256
- [88, 256, 476, 897], 241
- [241, 256, 476, 897], 88

Now we have 5 rows of data to test for the removed player.

Pre-Processing the Data

After preprocessing the data, this is what it looks like. This is what **mapped_expnaded.csv** looks like.

Which leaves us with **1,184,560 rows**

game	season	home_team	away_team	starting_min	outcome	home_players	away_players	player_removed
9610	8	29	10	0	-1	[88, 241, 256, 476]	[51, 307, 396, 617, 958]	897
9610	8	29	10	7	-1	[88, 241, 256, 897]	[307, 355, 396, 617, 958]	476
9610	8	29	10	8	-1	[241, 256, 785, 897]	[43, 355, 617, 713, 958]	138
9610	8	29	10	9	-1	[138, 241, 277, 785]	[43, 355, 617, 713, 958]	256
9610	8	29	10	10	-1	[138, 277, 785, 859]	[43, 355, 617, 713, 958]	843

Next, we filter only for games with outcome == 1. We want to train our model to predict for 'player_removed'.

Which leaves us with **440,875 rows**

game	season	home_team	away_team	starting_min	outcome	home_players	away_players	player_removed
9610	8	29	10	11	1	[277, 785, 843, 859]	[43, 355, 617, 652, 713]	138
9610	8	29	10	13	1	[277, 785, 843, 859]	[43, 51, 573, 652, 713]	138
9610	8	29	10	20	1	[88, 138, 256, 897]	[307, 396, 617, 795, 958]	241
9610	8	29	10	26	1	[88, 241, 256, 476]	[307, 355, 396, 617, 958]	897
9610	8	29	10	29	1	[138, 241, 785, 897]	[43, 307, 617, 795, 958]	476

Now we are ready to train the model.

Designing the Model: Feature Design

We decided to build a MLP, since the way we have preprocessed our data makes it a prediction problem.

Let's figure out what the input and output tensor shapes will be.

Input		Input		Input		Input		Input		Target Output
game	season	home_team	away_team	starting_min	outcome	home_players		away_players		player_removed
9610	8	29	10	11	1	[277, 785, 843, 859]		[43, 355, 617, 652, 713]		138
9610	8	29	10	13	1	[277, 785, 843, 859]		[43, 51, 573, 652, 713]		138
9610	8	29	10	20	1	[88, 138, 256, 897]		[307, 396, 617, 795, 958]		241
9610	8	29	10	26	1	[88, 241, 256, 476]		[307, 355, 396, 617, 958]		897
9610	8	29	10	29	1	[138, 241, 785, 897]		[43, 307, 617, 795, 958]		476

Input

- Season (8 → 2013)
- Home Team (29 → "PHO")
- Away Team (10 → "DEN")
- Home Players (length 4)
- Away Players (length 5)

6 tensors with shapes (1, 1, 1, 4, 5)

Target Output (what the model is predicting)

- Player_Removed

1 tensor with shape (1044)

Designing the Model

Summary So Far:

We are training the model on **440,875 rows** of data. Each of these rows is part of a match where the outcome == 1. In each row, a player is removed from the home_players list. The model's goal is to predict the player_removed.

Goal: Training the Model

Now we want to train the model using a MLP that predicts the player_removed element. Now that we successfully indexed our data, they can be embedded.

Model Design

1. Embed the Season data, home_team and away_team data, and Player (home & away) data to size 64 vector.
2. Set up the (season, startming_min) as the context feature
3. Set up the (home_team and away_team) as a teams feature
4. Set up the home_players and away_players as their own features
5. Combine all features, apply ReLU and dropout
6. Return Logits

Designing the Model

We are training the model on **440,875 rows** of data. Each of these rows is part of a match where the outcome == 1. In each row, a player is removed from the home_players list. The model's goal is to predict the player_removed.

Step 1: Using Embeddings, we can represent complex characteristics in categorical data.

3 Embedding Layers:

- Season Embedding
- Player Embedding
- Team Embedding

Input

- Season (Pass into Season Embedding)
- Home Team (Pass into Team Embedding)
- Away Team (Pass into Team Embedding)
- Home Players (Pass into Player Embedding)
- Away Players (Pass into Player Embedding)

Output

- Season (vector size 64)
- Home Team (vector size 64)
- Away Team (vector size 64)
- Home Players
- Away Players

Embedded Layer Definitions

```
self.player_embedding = nn.Embedding(len(names), embedding_dim)
self.season_embedding = nn.Embedding(len(seasons), embedding_dim)
self.team_embedding = nn.Embedding(len(teams), embedding_dim)
```

Forward Pass

```
season_emb = self.season_embedding(season_idx)
home_team_emb = self.team_embedding(home_team_idx)
away_team_emb = self.team_embedding(away_team_idx)

home_player_embeddings = self.player_embedding(home_players)
home_player_embeddings = home_player_embeddings.view(batch_size, -1)

away_player_embeddings = self.player_embedding(away_players)
away_player_embeddings = away_player_embeddings.view(batch_size, -1)
```

Designing the Model

Step 2: Once our data is embedded, we use Feature Layers to transform them into higher-level features useful for predicting. We broke features up in order to achieve domain-specific patterns.

4 Feature Processing Linear Layers

- Home Players FC ($4 \times 64 \rightarrow 128$)
- Away Players FC ($5 \times 64 \rightarrow 128$)
- Teams FC ($2 \times 64 \rightarrow 128$)
- Context (season, starting_min) FC ($64 + 1 \rightarrow 128$)

Input

- 4×64 Home Players neurons (from the Embedding Layers)
- 5×64 Away Players neurons (from the Embedding Layers)
- 2×64 Team neurons (from the Embedding Layers)
- $64 + 1$ Context neurons (from the Embedding Layers)

Output

- 128 neurons representing the 4 home players
- 128 neurons representing the 5 away players
- 128 neurons representing home team and away team
- 128 neurons representing the (season, starting_min)

Feature Layer Definitions

```
self.home_players_fc = nn.Linear(embedding_dim * 4, hidden_dim)
self.away_players_fc = nn.Linear(embedding_dim * 5, hidden_dim)
self.teams_fc = nn.Linear(embedding_dim * 2, hidden_dim)
self.context_fc = nn.Linear(embedding_dim + 1, hidden_dim)
```

Forward Pass

```
# 2. Features
home_player_features = F.relu(self.home_players_fc(home_player_embeddings))
away_player_features = F.relu(self.away_players_fc(away_player_embeddings))

context_features = torch.cat([
    season_emb,
    starting_min.unsqueeze(1)
], dim=1)
context_features = F.relu(self.context_fc(context_features))

teams_features = torch.cat([
    home_team_emb,
    away_team_emb
], dim=1)
teams_features = F.relu(self.teams_fc(teams_features))
```

Designing the Model

Step 3: This is the Combining Features Layer, where we combine all features to create an output.

Now that we have the following features from Step 2:

- Home Player Features (captures how the home players interact with each other) = 128 neurons
- Away Player Features (captures how the away players interact with each other) = 128 neurons
- Team Features (captures how the home_team interacts with the away_team) = 128 neurons
- Context Features (season and starting_min captures how the current time affects the model) = 128 neurons

Total neurons = $128 * 4 = 512$

Combination Layer 1:

1. We can capture the 512 neurons and break it down to 256 neurons.
2. Apply ReLU (to allow the model to learn complex relationships)
3. Apply Dropout (turns off 30% random neurons during training to prevent the model from overfitting)

Combination Layer 2:

1. Capture 256 neurons and break it down to 128 neurons.
2. Apply ReLU
3. Apply Dropout

Output Layer:

1. Capture 128 neurons and output to 1044 neurons

1044 neurons = 1044 unique names to which the model could turn on as the “predicted” player that was removed.

Combined Layer Definitions

```
self.combined_fc1 = nn.Linear(hidden_dim * 4, hidden_dim * 2)
self.combined_fc2 = nn.Linear(hidden_dim * 2, hidden_dim)
self.output_fc = nn.Linear(hidden_dim, len(names))
```

Forward Pass

```
# 3. Combined Features
combined_features = torch.cat([
    context_features,
    teams_features,
    home_player_features,
    away_player_features
], dim=1)

combined_features = F.relu(self.combined_fc1(combined_features))
combined_features = self.dropout(combined_features)
combined_features = F.relu(self.combined_fc2(combined_features))
combined_features = self.dropout(combined_features)

logits = self.output_fc(combined_features)
```

Designing the Model

Forward Pass

Layer Definitions

```
class BasketballModelMLP(nn.Module):
    def __init__(self, embedding_dim, hidden_dim):
        super(BasketballModelMLP, self).__init__()

        self.player_embedding = nn.Embedding(len(names), embedding_dim)
        self.season_embedding = nn.Embedding(len(seasons), embedding_dim)
        self.team_embedding = nn.Embedding(len(teams), embedding_dim)

        self.home_players_fc = nn.Linear(embedding_dim * 4, hidden_dim)
        self.away_players_fc = nn.Linear(embedding_dim * 5, hidden_dim)
        self.teams_fc = nn.Linear(embedding_dim * 2, hidden_dim)
        self.context_fc = nn.Linear(embedding_dim + 1, hidden_dim)

        self.combined_fc1 = nn.Linear(hidden_dim * 4, hidden_dim * 2)
        self.combined_fc2 = nn.Linear(hidden_dim * 2, hidden_dim)
        self.output_fc = nn.Linear(hidden_dim, len(names))

        self.dropout = nn.Dropout(0.3)
```

Why 64 Embedded Dimensions and 128 Hidden Dimensions?
Because:

- Too large = overfitting
- Too small = underfitting
- Power of 2 can be efficient sometimes

```
def forward(self, season_idx, home_team_idx, away_team_idx, starting_min, home_players, away_players):
    batch_size = season_idx.size(0)

    season_idx = season_idx.long()
    home_team_idx = home_team_idx.long()
    away_team_idx = away_team_idx.long()
    home_players = home_players.long()
    away_players = away_players.long()

    # 1. Embeddings
    season_emb = self.season_embedding(season_idx)
    home_team_emb = self.team_embedding(home_team_idx)
    away_team_emb = self.team_embedding(away_team_idx)

    home_player_embeddings = self.player_embedding(home_players)
    home_player_embeddings = home_player_embeddings.view(batch_size, -1)

    away_player_embeddings = self.player_embedding(away_players)
    away_player_embeddings = away_player_embeddings.view(batch_size, -1)

    # 2. Features
    home_player_features = F.relu(self.home_players_fc(home_player_embeddings))
    away_player_features = F.relu(self.away_players_fc(away_player_embeddings))

    context_features = torch.cat([
        season_emb,
        starting_min.unsqueeze(1)
    ], dim=1)
    context_features = F.relu(self.context_fc(context_features))

    teams_features = torch.cat([
        home_team_emb,
        away_team_emb
    ], dim=1)
    teams_features = F.relu(self.teams_fc(teams_features))

    # 3. Combined Features
    combined_features = torch.cat([
        context_features,
        teams_features,
        home_player_features,
        away_player_features
    ], dim=1)

    combined_features = F.relu(self.combined_fc1(combined_features))
    combined_features = self.dropout(combined_features)
    combined_features = F.relu(self.combined_fc2(combined_features))
    combined_features = self.dropout(combined_features)

    logits = self.output_fc(combined_features)

    return logits
```

Training the Model

Training Approach:

1. Forward Pass
2. Calculate the loss
3. Clear Gradients (optimizer.zero_grad())
4. Backward Pass
5. Optimizer Step (Apply gradients)

Optimizer: Adam (lr=0.001) → Adaptive Moment Estimator is a popular optimizer, great at working with embeddings because it adapts its learning rate for each parameter

Criterion: Cross Entropy Loss → good for classification problems

Epochs: 5

Batch Size: 32 (we used batch training)

Train/Test Split Ratio: 80%

With each epoch, we evaluated using an accuracy metric.

After 5 epochs of training, we achieved **37.18%** testing accuracy. Not to be confused with NBA_test.csv.

No existing model found. Starting with a new model.

```
Epoch 1/5: 100%|██████████| 11022/11022 [02:55<00:00, 62.84it/s]
Epoch 1/5, Train Loss: 3.3154, Test Loss: 2.4511, Accuracy: 27.07%
Epoch 2/5: 100%|██████████| 11022/11022 [02:48<00:00, 65.42it/s]
Epoch 2/5, Train Loss: 2.6466, Test Loss: 2.2110, Accuracy: 32.83%
Epoch 3/5: 100%|██████████| 11022/11022 [02:56<00:00, 62.50it/s]
Epoch 3/5, Train Loss: 2.4858, Test Loss: 2.0900, Accuracy: 35.03%
Epoch 4/5: 100%|██████████| 11022/11022 [02:57<00:00, 62.00it/s]
Epoch 4/5, Train Loss: 2.3987, Test Loss: 2.0285, Accuracy: 35.86%
Epoch 5/5: 100%|██████████| 11022/11022 [02:59<00:00, 61.23it/s]
Epoch 5/5, Train Loss: 2.3455, Test Loss: 1.9789, Accuracy: 37.18%
```

```
1 def train_model(model, train_dataloader, test_dataloader, epochs, learning_rate, device):
2     loss_fn = nn.CrossEntropyLoss()
3     optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
4
5     train_losses = []
6     test_losses = []
7
8     for epoch in range(epochs):
9         model.train()
10        epoch_loss = 0
11
12        for batch_idx, (context, target) in enumerate(tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}")):
13            context = context.to(device)
14            target = target.to(device).long().squeeze()
15
16            season_idx = context[:, 1]
17            home_team_idx = context[:, 2]
18            away_team_idx = context[:, 3]
19            home_players = context[:, 4:8]
20            away_players = context[:, 9:14]
21            starting_min = context[:, -2].float()
22
23            logits = model(season_idx, home_team_idx, away_team_idx, starting_min, home_players, away_players)
24            loss = loss_fn(logits, target)
25
26            optimizer.zero_grad()
27            loss.backward()
28            optimizer.step()
29
30            epoch_loss += loss.item()
31
32        avg_train_loss = epoch_loss / len(train_dataloader)
33        train_losses.append(avg_train_loss)
34
35        model.eval()
36        epoch_test_loss = 0
37        correct = 0
38        total = 0
39
40        with torch.no_grad():
41            for context, target in test_dataloader:
42                context = context.to(device)
43                target = target.to(device).long().squeeze()
44
45                season_idx = context[:, 1]
46                home_team_idx = context[:, 2]
47                away_team_idx = context[:, 3]
48                home_players = context[:, 4:8]
49                away_players = context[:, 9:14]
50                starting_min = context[:, -2].float()
51
52                logits = model(season_idx, home_team_idx, away_team_idx, starting_min, home_players, away_players)
53                loss = loss_fn(logits, target)
54
55                epoch_test_loss += loss.item()
56
57                _, predicted = torch.max(logits, 1)
58                total += target.size(0)
59                correct += (predicted == target).sum().item()
60
61        avg_test_loss = epoch_test_loss / len(test_dataloader)
62        test_losses.append(avg_test_loss)
63
64        accuracy = 100 * correct / total
65
66        print(f"Epoch {epoch+1}/{epochs}, Train Loss: {avg_train_loss:.4f}, Test Loss: {avg_test_loss:.4f}, Accuracy: {accuracy:.2f}%")
67
68    return train_losses, test_losses
```


Training the Model

1ST ITERATION out of 3

Some analytics:

- At Epoch = 2, we see a substantial gap decrease
- Test loss and train loss continued to decrease over time

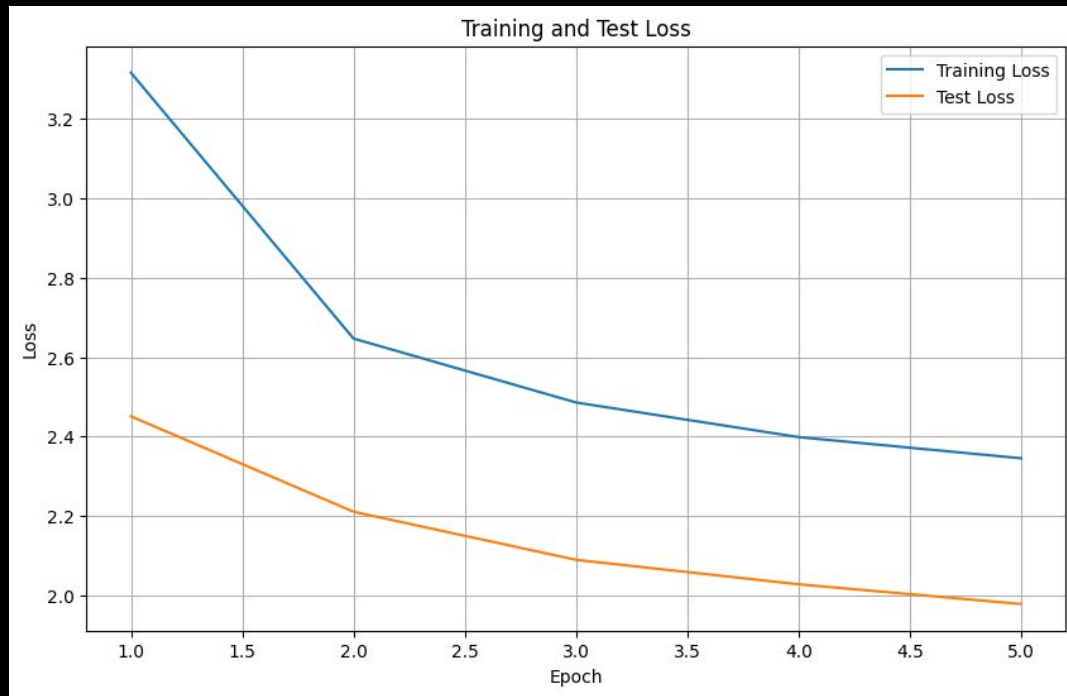
Results:

Model training time: ~12min 45s

Testing Accuracy: 37.18%

Final Test Loss: 1.9789

```
No existing model found. Starting with a new model.
Epoch 1/5: 100%|██████████| 11022/11022 [02:55<00:00, 62.84it/s]
Epoch 1/5, Train Loss: 3.3154, Test Loss: 2.4511, Accuracy: 27.07%
Epoch 2/5: 100%|██████████| 11022/11022 [02:48<00:00, 65.42it/s]
Epoch 2/5, Train Loss: 2.6466, Test Loss: 2.2110, Accuracy: 32.83%
Epoch 3/5: 100%|██████████| 11022/11022 [02:56<00:00, 62.50it/s]
Epoch 3/5, Train Loss: 2.4858, Test Loss: 2.0900, Accuracy: 35.03%
Epoch 4/5: 100%|██████████| 11022/11022 [02:57<00:00, 62.00it/s]
Epoch 4/5, Train Loss: 2.3987, Test Loss: 2.0285, Accuracy: 35.86%
Epoch 5/5: 100%|██████████| 11022/11022 [02:59<00:00, 61.23it/s]
Epoch 5/5, Train Loss: 2.3455, Test Loss: 1.9789, Accuracy: 37.18%
```



How can we improve the model training?

- Increasing # of epochs might cause overfitting, since we see the acceleration of accuracy plateau.
- Maybe we can try increasing the # of neurons?

Training the Model

2ND ITERATION out of 3

We tried doubling internal dimensions

- Embedding dimension 64 $\rightarrow$ 128
- Hidden dimension 128 $\rightarrow$ 256

Previous Results:

Model training time: ~12min 45s

Testing Accuracy: 37.18%

Final Test Loss: 1.9789

Results:

Model training time: ~38 min

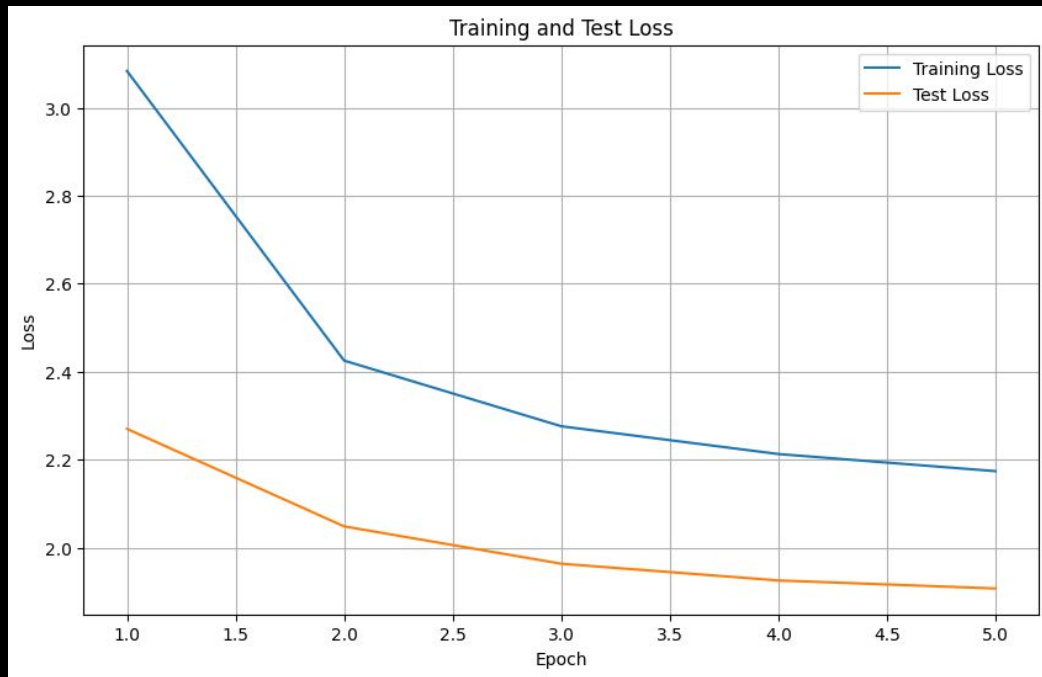
Testing Accuracy: 37.83%

Final testing loss: 1.9073

```
Epoch 1/5: 100%|██████████| 11022/11022 [07:09<00:00, 25.65it/s]
Epoch 1/5, Train Loss: 3.0834, Test Loss: 2.2700, Accuracy: 31.05%
Epoch 2/5: 100%|██████████| 11022/11022 [07:29<00:00, 24.52it/s]
Epoch 2/5, Train Loss: 2.4252, Test Loss: 2.0485, Accuracy: 36.18%
Epoch 3/5: 100%|██████████| 11022/11022 [07:40<00:00, 23.91it/s]
Epoch 3/5, Train Loss: 2.2760, Test Loss: 1.9636, Accuracy: 37.18%
Epoch 4/5: 100%|██████████| 11022/11022 [07:57<00:00, 23.07it/s]
Epoch 4/5, Train Loss: 2.2128, Test Loss: 1.9255, Accuracy: 37.86%
Epoch 5/5: 100%|██████████| 11022/11022 [08:39<00:00, 21.21it/s]
Epoch 5/5, Train Loss: 2.1739, Test Loss: 1.9073, Accuracy: 37.83%
```

We increased the model training time from 12min45s to ~38 min, and only achieved a minor accuracy bump and a slightly lower test loss.

We concluded that this wasn't the solution.



Training the Model

We adjusted the Embedding Dim and Hidden Dim back to 64 and 128.

Then we decreased dropout from 0.3 to 0.25

Results:

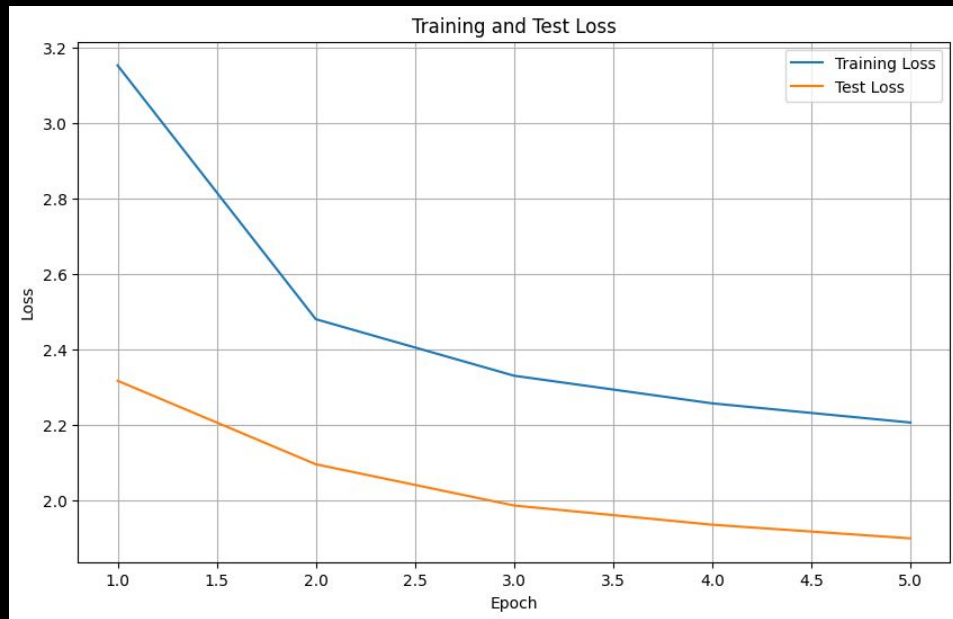
- Model training time: ~14min
- Testing Accuracy: 39.70%
- Final Test Loss: 1.8995

We were happy with the lower test loss.

We believe we could've stopped training at 3 epochs.

3RD ITERATION out of 3

```
No existing model found. Starting with a new model.  
Epoch 1/5: 100%|██████████| 11022/11022 [03:25<00:00, 53.69it/s]  
Epoch 1/5, Train Loss: 3.1532, Test Loss: 2.3173, Accuracy: 30.55%  
Epoch 2/5: 100%|██████████| 11022/11022 [03:24<00:00, 53.80it/s]  
Epoch 2/5, Train Loss: 2.4804, Test Loss: 2.0959, Accuracy: 35.20%  
Epoch 3/5: 100%|██████████| 11022/11022 [03:40<00:00, 50.03it/s]  
Epoch 3/5, Train Loss: 2.3306, Test Loss: 1.9866, Accuracy: 37.88%  
Epoch 4/5: 100%|██████████| 11022/11022 [03:39<00:00, 50.32it/s]  
Epoch 4/5, Train Loss: 2.2573, Test Loss: 1.9356, Accuracy: 38.89%  
Epoch 5/5: 100%|██████████| 11022/11022 [03:46<00:00, 48.57it/s]  
Epoch 5/5, Train Loss: 2.2065, Test Loss: 1.8995, Accuracy: 39.70%
```



Testing the Model

Using our 3rd iteration of the model, we achieved the following results:

- **28.90%** overall accuracy → The model successfully guessed the right 5th player 28.90% of the time.
- The 5th player was in the **Top 10** candidates **73.00%** of the time
- The 5th player was in the **Top 5** candidates **65.30%** of the time
- The 5th player was in the **Top 3** candidates **53.90%** of the time

See [NBA_Testing.ipynb](#). This jupyter notebook created a final CSV of all predictions.

predicted_player	confidence	true_player	true_player_rank	true_player_probability	correct
Jermaine O'Neal	36.40018701553345	Troy Murphy	3.0	14.056769371032715	False
Chuck Hayes	25.059589743614197	Chuck Hayes	1.0	25.059589385986328	True
Manu Ginobili	22.772815823554993	Brent Barry	2.0	18.929443359375	False
Kevin Garnett	35.5131596326828	Craig Smith	3.0	22.162633895874023	False
Darko Milicic	96.0522711277008	Stromile Swift	14.0	0.002492165192961693	False
Charlie Bell	27.6822030544281	Charlie Bell	1.0	27.68220329284668	True
Dwyane Wade	26.970288157463074	Dwyane Wade	1.0	26.970287322998047	True
Anderson Varejao	72.23473787307739	Anderson Varejao	1.0	72.2347412109375	True
Stephen Jackson	31.178978085517883	Mickael Pietrus	3.0	12.622146606445312	False

Some Things We Noticed

Some data observations:

- There are 40 new names in testing data in NBA_test.csv. The model has no prior data to these 40 new names, so it cannot make guesses on if it is good with other team members or if it should be a 5th player.
- There is a 10th season: NBA_test.csv has a 10th season: Season 2016. There is no past data on 2016, so predictions on 2016 games are unpredictable on who the model will recommend as the 5th player.

- 2 names that are not in the training data or the test data. NBA_test_labels.csv introduces two new names: “Anthony Brown” and “R.J. Hunter”, which are not found anywhere in ‘NBA_test.csv’ and ‘matchups-20*.csv’. It is impossible for the model to ever get this prediction right, because it is a completely new name, as if they weren’t even NBA players.

40 names that were found in NBA_test.csv but did not exist in matchups-20*.csv

```
names: ['Alan Williams', 'Larry Nance', 'Bobby Portis', 'Nikola Jokic', 'Christian Wood', 'Norman Powell', 'Tyus Jones', 'Frank Kaminsky', 'Devin Booker', 'Cameron Payne', 'Kristaps Porzingis', 'Josh Richardson', 'Emmanuel Mudiay', 'Nemanja Bjelica', 'Stanley Johnson', 'Jahlil Okafor', 'D'Angelo Russell', 'Willie Cauley-Stein', 'Karl-Anthony Towns', 'Justise Winslow', 'T.J. McConnell', 'Myles Turner', 'Jordan McRae', 'Terry Rozier', 'Marcelo Huertas', 'Jonathon Simmons', 'Rondae Hollis-Jefferson', 'Mario Hezounja', 'Montrezl Harrell', 'Willie Reed', 'Lamar Patterson', 'Xavier Munford', 'Kelly Oubre', 'Raul Neto', 'Richaun Holmes', 'Trey Lyles', 'Chris McCullough', 'Rashad Vaughn', 'DeLon Wright', 'Jerian Grant']
Total unique names: 1084 | First 5 names: ['A.J. Price', 'Aaron Brooks', 'Aaron Gordon', 'Aaron Gray', 'Aaron McKie']
```

Testing the Model

Why we think our prediction accuracy is low:

On row 941: Our model was 44.44% confident that “Jordan Farmer” was the 5th optimal player, but the true answer was Anthony Brown. Since Anthony Brown’s name is not an output neuron at all (since his name does not exist in any of the training data), this was a failed attempt from the start.

Jordan Farmer	44.448599219322205	Anthony Brown		
---------------	--------------------	---------------	--	--

On row 910: Our model was 52.69% confident that “Luke Ridnour” was the 5th optimal player, but the true answer was Shabazz Muhammad. It was completely wrong because the Away Team included Alan Williams which was one of the 40 names that was not included in the Training Data.

Luke Ridnour	52.69308090209961	Shabazz Muhammad	421.0	2.7463205320987782e-12	False
--------------	-------------------	------------------	-------	------------------------	-------

['Alan Williams', 'Chase Budinger', 'Devin Booker', 'Jon Leuer', 'Ronnie Price']

Testing the Model

Why we think our prediction accuracy is low (part 2):

Training data only contained seasons 2007-2015

Testing data contained season 2016

The true player ranking standard deviation is substantially higher in 2016 than in the other seasons.

This is expected, since the data our model was trained on is only for season 2007-2015



```
1 season_std_devs = results_df.groupby('season')['true_player_rank'].std()  
2 print("Standard Deviation of True Removed Player Rank by Season:")  
3 print(season_std_devs)
```



Standard Deviation of True Removed Player Rank by Season:

season	
2007	43.070724
2008	30.179262
2009	9.848222
2010	28.969570
2011	38.944367
2012	83.959015
2013	47.577317
2014	18.266874
2015	32.809304
2016	216.385154

Name: true_player_rank, dtype: float64

Future Improvements

Model doesn't take into account if the predicted home_player is part of the home_team!

- An improvement to this is to build a dictionary of valid answers, then have the model choose from the list of valid players.
- But we believe that since the model is trained on past data, that it predicts based on who is part of the valid team anyways, just by nature. To elaborate, the model is trained on data from 2013 games, for example, and therefore is trained to make predictions on the 5th player based on the 2013 games. So if you gave it a home team of 4 players from the 2013 season, it is trained to pick the 5th player based on who was on the 2013 season anyways.